Monte Carlo Tree Search for Hex and Y

Classical UCT–RAVE, Gumbel AlphaZero, and a Unified Y Extension

Lea Dodeman Nestor Antunano Cabrera

April 8, 2026

Contents

1 Introduction **2**

2 Games and Scope **2**
2.1 Hex . . . 2
2.2 Y . . . 2
2.3 Repository Modules . . . 3

3 Search Methods **3**
3.1 Classical MCTS with UCT and RAVE . . . 3
3.2 Gumbel AlphaZero . . . 3

4 Unified Implementation Strategy **3**
4.1 Reference Path and Fast Path . . . 4
4.2 Incremental Connectivity . . . 4
4.3 Cython Acceleration . . . 4

5 HexClassic **4**
5.1 Goal . . . 4
5.2 Results . . . 4

6 HexGumbel **5**
6.1 Method . . . 5
6.2 Results . . . 6

7 YClassic **6**
7.1 Design Goal . . . 6
7.2 Rule Model and Board Logic . . . 6
7.3 Search Adaptation . . . 7
7.4 Verification . . . 7
7.5 Performance . . . 7
7.6 Full-Scale Experimental Results . . . 7

8 Discussion **8**

9 Conclusion **9**

Abstract

This repository studies Monte Carlo Tree Search across two connection games, Hex and Y, inside one shared implementation framework. The project has three active branches: a classical Hex baseline reproducing the UCT+RAVE setting of *Monte-Carlo Hex*, a learned Hex agent based on Gumbel AlphaZero, and a new classical implementation for Y. The main engineering goal is consistency: the classical branches expose the same Python and Cython APIs, use incremental connectivity tracking, and can be compared without conflating algorithmic differences with implementation asymmetries. The main research takeaway is twofold. First, the classical Hex results preserve the qualitative behavior reported in the literature even with simpler rollouts. Second, the new Y implementation now matches the Hex branch in logic and speed infrastructure. Its full-size experiments show the expected simulation-scaling curve, a strong preference for $C_{\text{uct}} = 0$, and a clear benefit from connectivity-aware rollouts, while also making clear that Y still lacks an analogue of Hex bridge knowledge with equally strong structural impact.

1 Introduction

Monte Carlo Tree Search (MCTS) is a search procedure that incrementally builds a partial game tree through four repeated phases: selection, expansion, simulation, and backpropagation. Rather than expanding all children uniformly, it allocates simulations to the lines of play that appear most promising under the current statistics. This makes MCTS particularly effective in deterministic perfect-information games with large branching factors and difficult static evaluation functions.

This repository began as a Hex project and then expanded into a broader study of connection-game search. It now contains:

- `HexClassic/`: a classical UCT+RAVE implementation inspired by Cazenave and Saffidine's *Monte-Carlo Hex* [2],
- `HexGumbel/`: a neural-network-guided Hex agent using the Gumbel AlphaZero root policy [3],
- `YClassic/`: a classical Y implementation designed to reach the same logic and performance standard as `HexClassic/`.

The documentation is organized around the repository as a whole rather than as three disconnected folders. Shared engineering choices are described once, then each branch is discussed only where its algorithmic behavior or game-specific logic diverges.

2 Games and Scope

2.1 Hex

Hex is played on an $N \times N$ rhombus of hexagonal cells. One player tries to connect top to bottom, while the other tries to connect left to right. Players alternately occupy empty cells, and stones are never moved or removed. Hex has no draws on a full board, which makes it a clean fit for simulation- based search.

2.2 Y

Y is played on a triangular board with hex-style adjacency. A player wins by forming one connected component that touches all three sides of the triangle. Corner cells count as belonging to both adjacent sides, and a full board has exactly one winner [1, 4]. Standard descriptions of Y also note that the game can be played either on a regular triangular hex grid or on the "official"

Schensted/Titus board that inserts three five-neighbor points [1, 4]. This project uses the regular triangular grid because it is the simplest variant for a direct implementation comparison with Hex.

2.3 Repository Modules

Module	Game	Role
`HexClassic/`	Hex	Classical UCT+RAVE baseline and reproduction study
`HexGumbel/`	Hex	Learned policy-value search with Gumbel root search
`YClassic/`	Y	Classical Y search adapted from the Hex baseline

The Y work in the current repository is focused on the classical path only.

3 Search Methods

3.1 Classical MCTS with UCT and RAVE

The classical baseline follows the UCT+RAVE structure of [2]. Selection uses the empirical action value augmented by the UCT exploration bonus:

$$\mathrm{UCT}(s,a) = Q(s,a) + C_{\mathrm{uct}}\sqrt{\frac{\log N(s)}{N(s,a)}}.$$

RAVE (Rapid Action Value Estimation), also called AMAF, shares information between branches by treating moves played later in a simulation as if they had been tried immediately. The implementation interpolates between the Monte Carlo estimate and the RAVE estimate using the same bias-controlled blending scheme used in [2]. This is important in early search, where raw visit counts are still sparse.

3.2 Gumbel AlphaZero

The learned Hex branch replaces rollout evaluation with a neural policy-value model. At interior nodes it uses the PUCT rule

$$Q(s,a) + c_{\mathrm{puct}} P(s,a) \frac{\sqrt{N(s)}}{1 + N(s,a)},$$

and at the root it follows the Gumbel AlphaZero policy-improvement mechanism of [3]. Legal moves receive Gumbel perturbations, the root policy is filtered to a candidate set, and sequential halving allocates the simulation budget among the strongest contenders. Relative to a pure AlphaZero pipeline [5], the project adds supervised pretraining from classical Hex labels because the available compute budget is limited.

4 Unified Implementation Strategy

The repository uses one shared engineering pattern across the classical branches, and this is the main reason the new Y implementation could be brought up to the same standard as the older Hex code quickly.

4.1 Reference Path and Fast Path

Each classical module exposes:

- a pure Python reference implementation that is easy to inspect,
- a Cython implementation with the same public API for serious experimentation.

This keeps the algorithm readable while still making large experimental runs practical.

4.2 Incremental Connectivity

Both Hex and Y are connection games, so the critical inner-loop operation is not piece movement but component maintenance. The repository therefore tracks connectivity incrementally with Union-Find structures rather than recomputing graph connectivity after every move.

In Hex, components connect to two virtual borders. In Y, components instead carry a three-bit side mask. A Y move wins as soon as the merged component mask becomes `111`, meaning that the connected component touches left, right, and bottom simultaneously.

4.3 Cython Acceleration

The fast classical backends follow the same optimization pattern once and only once across the whole project:

- board state in typed C arrays,
- board cloning with `memcpy`,
- pooled MCTS nodes rather than Python object churn,
- flat RAVE arrays instead of Python dictionaries in hot loops,
- explicit RNG seeding for multiprocessing workers.

The point is not that Hex and Y require two different stories about why Cython helps. They benefit from the same structural reduction in interpreter overhead.

5 HexClassic

5.1 Goal

`HexClassic/` is a comparative study rather than an attempt to recreate the strongest known Hex engine. It asks whether the qualitative findings of *Monte-Carlo Hex* still hold if the rollout policy is simplified from the paper's type-3 setting to type-2 bridge defense only.

5.2 Results

All results below use the Cython backend on 11×11 boards with 200 games per data point.

Simulation count.

Simulations	Ours	Paper	Delta
1,000	31.5%	6.0%	+25.5
2,000	45.5%	11.5%	+34.0
4,000	41.5%	20.0%	+21.5
8,000	50.0%	33.0%	+17.0
32,000	55.5%	61.0%	-5.5
64,000	66.5%	68.5%	-2.0

The expected monotone trend is preserved. The weaker rollout policy compresses the low-simulation part of the curve, but the results converge toward the paper once the tree policy dominates the playout quality.

UCT exploration constant.

C_{uct}	Ours	Paper	Delta
0.0	88.0%	61.0%	+27.0
0.1	62.5%	60.0%	+2.5
0.2	52.5%	55.5%	-3.0
0.4	43.0%	42.0%	+1.0
0.5	55.5%	41.0%	+14.5
0.6	47.5%	35.5%	+12.0
0.7	50.5%	32.5%	+18.0

The strongest setting remains $C_{\text{uct}} = 0$ when RAVE is enabled. This matches the interpretation from [2]: RAVE is already doing the work of early exploration.

Rollout policy and RAVE bias.

Experiment	Result	Interpretation
Random vs bridge rollouts	28.5%	Bridges clearly stronger than random
$b = 0.0005$ vs 0.001	47.0%	
$b = 0.00025$ vs 0.001	49.0%	RAVE tuning is flatter than in the paper
$b = 0.000125$ vs 0.001	46.0%	

The main conclusion is that the UCT+RAVE core remains robust even when the rollout policy is intentionally simplified.

6 HexGumbel

6.1 Method

`HexGumbel/` uses a ResNet that outputs both a policy over legal moves and a scalar value estimate. Classical rollouts disappear; the search is guided by the network and by the Gumbel AlphaZero root policy [3]. The project uses a supervised warm start from `HexClassic/` labels before entering self-play. This makes the training pipeline more stable than a pure cold-start AlphaZero setup under limited compute.

6.2 Results

The learned agent is evaluated on 7×7 Hex with only 16 simulations per move against a classical Hex baseline using 16,000 simulations per move.

Iteration	vs Random	vs MCTS 16K	Buffer Size
5	100%	10.0%	69,220
10	100%	32.5%	132,931
20	100%	45.0%	255,492
30	100%	55.0%	121,818
40	100%	62.5%	63,061
50	100%	55.0%	61,491
60	100%	50.0%	62,208
70	100%	70.0%	188,366

The learned agent eventually reaches roughly 65–70% against the classical baseline despite using a thousand times fewer simulations. The result is not that search became unnecessary; it is that a better prior and a learned value allow the search budget to be deployed much more selectively.

7 YClassic

7.1 Design Goal

The Y branch started as a draft and lacked the implementation maturity of `HexClassic/`. The goal was therefore not just to make it run, but to bring it to the same standard in three dimensions:

- rule correctness,
- architectural parity with the Hex baseline,
- practical speed through a Cython backend.

7.2 Rule Model and Board Logic

The regular-triangle Y implementation uses

$$n = \frac{s(s+1)}{2}$$

cells for side length s. Each move belongs to zero, one, or two sides of the triangle depending on whether it lies in the interior, on an edge, or in a corner. For each player, every connected component stores a three-bit side mask:

$$\texttt{LEFT} = 001, \quad \texttt{RIGHT} = 010, \quad \texttt{BOTTOM} = 100.$$

Whenever a new stone is placed, it merges with same-color neighboring components and the masks are OR-combined. A player wins as soon as one merged component reaches `111`. This makes Y win detection constant-time after each move.

7.3 Search Adaptation

Y does not have a direct analogue of Hex bridge templates. The classical Y rollout policy therefore uses a local connectivity score rather than a fixed bridge-save response. Candidate rollout moves are preferred when they:

- merge multiple friendly components,
- increase the number of sides touched by the merged component,
- occupy strategically relevant cells near enemy groups.

The search core itself remains the same as in Hex: one UCT+RAVE selection loop, AMAF backpropagation, and full-board playouts.

7.4 Verification

The Y implementation was checked at three levels:

1. exhaustive full-board checks on small triangular boards to confirm that every coloring has exactly one winner,
2. random-game checks against a naive connectivity evaluator,
3. Python-vs-Cython consistency checks on identical move sequences.

The automated regression suite currently passes in both Python-only mode and with the compiled Cython backend.

7.5 Performance

An empty-board benchmark on side length 7 with 2,000 simulations produced the following move-selection times:

Backend	Time
Python	0.298 s
Cython	0.0135 s
Speedup	22.1×

This closes the original engineering gap between the Y draft and the mature Hex implementation. On the larger side-length-11 benchmark with the Cython backend and the same 2,000-simulation root search, Hex took 0.0276 s and Y took 0.0377 s. In other words, Y is only about 1.37× slower than Hex in the matched non-toy setting.

7.6 Full-Scale Experimental Results

The full Y experiment suite was then run with the same protocol shape used in `HexClassic/`: Cython backend, side length 11, 200 games per point, and seed 42. The experiment runner saves one JSON artifact per table plus a consolidated `all_results.jsonl` file in `YClassic/results/`.

Simulation count.

Simulations	Win rate against 16,000-sim reference
1,000	2.5%
2,000	17.0%
4,000	34.0%
8,000	44.5%
32,000	55.0%
64,000	60.5%

This is the expected qualitative behavior: more simulations still mean stronger play, and the curve crosses the 50% mark once the search budget exceeds the 16,000-simulation reference.

UCT exploration constant.

C_{uct}	Win rate against $C_{\text{uct}} = 0.3$
0.0	90.0%
0.1	76.0%
0.2	59.0%
0.4	45.5%
0.5	43.5%
0.6	45.5%
0.7	47.5%

The Y branch shows the same qualitative pattern seen in Hex: when RAVE is active, the strongest setting is again $C_{\text{uct}} = 0$, and the preference is even more decisive here.

Rollout policy and RAVE bias.

Experiment	Result	Interpretation
Random vs connectivity rollout	37.5%	Connectivity is clearly stronger
$b = 0.0005$ vs 0.001	51.5%	
$b = 0.00025$ vs 0.001	50.0%	RAVE bias is comparatively flat
$b = 0.000125$ vs 0.001	47.5%	

The important point is that the Y branch no longer only "looks plausible" in small calibration tests. At the same non-toy 11-side scale used for the Hex study, it shows clear simulation scaling, a strong tree-policy preference, and a measurable gain from rollout knowledge. What remains flatter than in Hex is not the implementation quality, but the strength gap between lightweight Y heuristics and the stronger bridge knowledge available in Hex.

8 Discussion

Taken together, the three modules show three complementary points.

- Classical search remains competitive and interpretable in connection games, especially when paired with well-chosen rollout knowledge.
- Learned search can dominate rollout-based evaluation when the policy and value network are good enough, even under a much smaller simulation budget.

- Extending a mature search system to a new game is not only a matter of copying code. The board geometry, win condition, and available domain heuristics all matter. Y could inherit the Hex engineering backbone, but it still required new logic, and its heuristics remain weaker and less structured than Hex bridge knowledge even after the full-scale run.

The most important engineering lesson is that the shared architecture paid off. Once the repository had a consistent pattern for Python references, Cython fast paths, Union-Find connectivity, and reproducible experiments, the Y branch could be upgraded quickly without becoming a one-off implementation.

9 Conclusion

This repository now contains a coherent MCTS study across Hex and Y. The classical Hex branch reproduces the main qualitative findings of *Monte-Carlo Hex*. The learned Hex branch shows that Gumbel AlphaZero can outperform a far more heavily simulated classical baseline on the chosen 7x7 setting. The Y branch is now no longer a draft: it has correct rule handling, a matching Python/Cython structure, automated regression tests, and a compiled backend with a measured 22x speedup over the Python reference.

More importantly, the Y branch is now documented with real non-toy experiments rather than only pilot checks. On 11-side boards it shows the expected simulation-scaling curve, strongly prefers $C_{\text{uct}} = 0$ under RAVE, and its connectivity rollout defeats purely random playouts by a clear margin. Performance is also in the right regime: at matched 11-side, 2,000-simulation root search, Y is only about 1.37× slower than Hex.

The natural next step for Y is not more implementation cleanup but stronger domain knowledge: the search stack is ready, and the remaining open question is which rollout or prior knowledge best captures Y-specific structure.

References

[1] Boardspace.net. Game of y. https://www.boardspace.net/english/about_y.html, 2026. Accessed 2026-04-08.

[2] T. Cazenave and A. Saffidine. Monte-carlo hex. In *Computers and Games*, 2010.

[3] I. Danihelka, M. Schmid, D. Horgan, J. Schrittwieser, I. Antonoglou, L. Espeholt, T. Hubert, K. Simonyan, L. Sifre, T. Schaul, M. Lai, A. Guez, M. Lanctot, T. Graepel, D. Hassabis, and D. Silver. Policy improvement by planning with gumbel. *International Conference on Learning Representations*, 2022.

[4] Gambiter. Y - game. https://gambiter.com/connection/Y_game.html, 2026. Accessed 2026-04-08.

[5] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017.